DEEP RESEARCH REPORT

Dev-Docs Tooling for Agentic Workflows

Context7 internals, code-mode MCP, CLI-based retrieval, and llms.txt-native patterns - evaluated against published benchmarks, production measurements, and community evidence to answer one question: what should a documentation tool for coding agents look like?

Tooling Research & Architecture Evaluation

Prepared for internal build decision - normalized evidence, cost model, scenario verdicts

OCTOBER 2026

Contents

1. Executive Summary and Verdict . . . 1
2. Problem Space: Why Dev-Docs Context Is Broken . . . 2
The naive alternative: bigger prompts and cached context . . . 2
3. Anatomy of Context7: Reconstructed Internals . . . 3
The ingestion pipeline . . . 3
The serving contract . . . 4
Scale, freshness, and pricing history . . . 5
Documented criticisms . . . 5
What the open/closed split means for builders . . . 5
4. The Context Tax: Measured MCP Costs . . . 6
The accuracy side of the tax . . . 7
Why schemas cost so much . . . 8
5. Code Mode: MCP as Code APIs . . . 9
Where code mode underperforms . . . 10
6. The CLI Path: The gh-cli Hypothesis, Tested . . . 10
Prior art: the docs-CLI lineage . . . 11
Design rules for a docs CLI . . . 12
7. Docs-Native Patterns: llms.txt, Skills, and Repo Maps . . . 12
Progressive disclosure: Agent Skills . . . 13
Budget discipline: the aider repo map . . . 13
8. Normalized Benchmark Comparison . . . 14
Methodology warnings . . . 14
The missing axis: latency . . . 15
9. Cost Model: Dollars per 1,000 Agent Queries . . . 15
Sensitivity analysis . . . 17
10. Scenario Matrix: Which Approach Wins Where . . . 17
Composition across scenarios . . . 18
11. Recommended Architecture: The Layered Docs Tool . . . 19
Layer rationale, mapped to evidence . . . 20
12. Build Decision, Risks, and Roadmap . . . 20

Risks and mitigations 21
90-day roadmap with validation gates 21
What would change the verdict 22
Sources 22

1. Executive Summary and Verdict

This report answers a practical question: if you want to build a Context7-style tool that puts official development documentation in front of coding agents, which delivery architecture should you build? Four candidates dominate the 2025-2026 discussion. The first is the Context7 model itself - a hosted ingestion pipeline that crawls, enriches, embeds, and re-ranks library documentation, then serves it through a two-tool MCP server. The second is the code-mode pattern popularized by Cloudflare and Anthropic, in which tool surfaces are compiled into typed code APIs that the agent scripts against instead of calling directly. The third is the CLI path, where documentation retrieval is just another shell verb with zero tool-schema overhead - the same reasoning that makes many teams prefer **gh** over the GitHub MCP server. The fourth is a docs-native pattern built directly on llms.txt, per-site MCP servers, and progressive-disclosure formats such as Agent Skills.

The evidence assembled here - engineering blog measurements, peer-reviewed benchmarks, issue-tracker telemetry, and community consensus - converges on a clear structural finding: **the delivery protocol is not where the value lives; the retrieval core is.** Raw MCP servers that dump dozens of tool schemas into context cost 17,600 to 80,000 tokens before an agent reads its first message [13][14][16], and measured setups with three servers consumed 143,000 of a 200,000-token window - 72 percent - on tool definitions alone [14]. Tool-selection accuracy measurably collapses as tool count grows past roughly 30 to 50 definitions [19][28]. Meanwhile, the same delivery need can be met with roughly 1,000 to 2,000 tokens when tools are deferred, compressed, or expressed as code [9][11][13], and with literally zero schema cost when exposed through a shell - which models already know how to drive from training [17][18][37].

26-55k	98.7%	13.6% - 43.1%	0
schema tokens for a raw GitHub MCP server, before any work [13][16][18]	token reduction from code execution over MCP (Anthropic) [9]	tool-selection accuracy without vs with retrieval pre-selection (RAG-MCP) [19]	tool-schema tokens consumed by a CLI such as gh [17][18]

The build verdict follows from that evidence and is developed quantitatively in Chapters 8 through 12. **Do not clone Context7's hosted service** - its economics depend on crawling infrastructure, and the January 2026 cut of its free tier from roughly 6,000 to 1,000 requests per month demonstrates how quickly the floor moved under its users [7][40]. **Do not build a wide, general-purpose MCP server** either - the context-tax literature shows that surface area is a liability, and platform vendors are already auto-managing it with tool search [27][28]. Instead, **build a layered, local-first docs tool**: a CLI with a SQLite full-text index over llms.txt-first ingestion, wrapped in a progressive-disclosure contract (name, then summary, then full snippet), and exposed through three thin adapters - a CLI as the primary interface, a minimal two-tool MCP shim for compatibility, and a typed code-mode module for sandboxed harnesses. Each layer of that recommendation is tied to cited evidence in Chapter 11, and a 90-day validation-gated roadmap appears in Chapter 12.

2. Problem Space: Why Dev-Docs Context Is Broken

Coding agents fail on library APIs for a structural reason: their parametric knowledge has a training cutoff, while the frameworks they are asked to use change weekly. Anthropic's original Context7 problem statement demonstrated this concretely - a then-current Claude model could not correctly write an **@upstash/redis** command that postdated its training data, even when the error message was pasted into the prompt [1]. Documentation is the fix, but documentation is also large, noisy, and versioned, which makes *how* it enters the context window a first-order engineering decision rather than a detail.

Three well-replicated results define the constraints. First, model performance degrades as input length grows even when the task is identical - Chroma's *Context Rot* study showed that increasing input tokens reliably degrades reasoning quality across models, so every token of irrelevant context is not neutral filler but active interference [23]. Second, models exhibit positional bias: relevant material buried in the middle of a long context is retrieved and used measurably worse than material at the edges - the *Lost in the Middle* result [24]. Third, Anthropic's context-engineering guidance frames attention as a finite budget to be spent deliberately, favoring just-in-time retrieval of small, high-signal chunks over pre-loading large corpora [26]. Together these results imply that a docs tool should optimize for **small, precise, positionally-sane injections**, not maximal recall.

The counterweight is that documentation genuinely helps code generation when retrieval works. CodeRAG-Bench, the largest systematic study of retrieval-augmented code generation, found that providing gold (oracle) documents yields significant pass-rate gains for GPT-4-class models across general programming, open-domain, and repository-level tasks, and that retrieving from larger, diverse open datastores can improve results even on top of GPT-4 [20]. The same study, however, documents a wide gap between oracle retrieval and what real retrievers achieve, especially when lexical overlap between query and document is low - retrieval quality, not willingness to use docs, is the bottleneck [20].

Finally, the economics have two distinct cost centers that are routinely conflated. **Schema bloat** is the fixed rent of tool definitions loaded on every turn; **response bloat** is the per-call cost of oversized tool outputs flowing through context [13]. A tool that returns 5,000 tokens of documentation per lookup can be more expensive than a tool whose schema costs 1,000 tokens, and vice versa - which is why this report normalizes and evaluates the two separately throughout. The tool you are building is, in short, a retrieval and context-budget problem wearing the costume of an integration problem.

The naive alternative: bigger prompts and cached context

Before engineering anything, it is worth ruling out the trivial solution. Anthropic's own contextual-retrieval paper opens with the honest observation that if a knowledge base is small enough - under roughly 200,000 tokens, about 500 pages - the best design is no retrieval at all: put the entire corpus in the prompt and let prompt caching absorb the cost, which cuts latency by more than half and input costs by up to 90 percent for repeated prefixes [21]. For a personal tool covering twenty libraries, llms-full.txt files plus prompt

caching is a legitimately competitive baseline, and any build decision should say why it loses. It loses on three measurable grounds as scope grows: cost scales linearly with corpus size on every cache miss; fresh content still has to be selected per task or the model drowns (context rot applies to documentation exactly as to anything else [23]); and version pinning becomes manual. The tool in this report is therefore justified by the 50-to-500-library case, where selective retrieval beats both naive stuffing and per-question web fetching.

3. Anatomy of Context7: Reconstructed Internals

Context7 is built and maintained by Upstash, the serverless data platform company, and was launched in early 2025 as a free utility for AI code editors before growing into a commercial developer platform with more than 100,000 indexed libraries [1][2][8]. Its GitHub repository - 63,700 stars at the time of writing - contains only the serving layer; the crawling engine, parsing engine, and API backend are deliberately closed-source [4]. Understanding what is open and what is closed matters directly for your build decision, because the open part is the thin layer and the closed part is where most of the engineering effort lives.

The ingestion pipeline

Upstash's launch posts describe a five-stage offline pipeline, reproduced in Figure 1. Documentation is **parsed** to extract code snippets and examples; **enriched** with short explanations and metadata generated by LLMs; **vectorized** into embeddings for semantic search; **re-ranked** with a custom relevance algorithm; and **cached** in Redis for low-latency serving [1][2]. Around this pipeline sits a quality plane that Upstash documents in its enterprise material: trust scores computed from repository signals (stars, activity, account age) and website signals (TLS, domain authority, backlinks); deduplication via exact-match and cosine-similarity filtering; version-aware parsing that stores the current docs version by default; a two-pass prompt-injection detection pipeline over ingested content; and a library benchmark system that generates developer-style questions and scores how well each library's docs answer them [6]. That benchmark-driven retrieval quality loop is one of the more under-appreciated parts of the design - it is what makes ranking an measured property rather than a heuristic.



Figure 1: Context7's reconstructed architecture - closed ingestion and quality planes feeding an open, two-tool MCP serving layer [1][4][5][6].

The serving contract

The public MCP server exposes exactly two tools, and their behavior has been documented from live Claude Code traces [2][5]. **resolve-library-id** takes a best-guess library name from the agent and returns ranked candidates, each with a Context7-compatible identifier in **/org/project** form, a short description, a code-snippet count, and a numeric trust score; the tool's own response text coaches the model to select on name match, trust score, snippet coverage, and relevance. **get-library-docs** takes the chosen identifier plus a topic string and a token budget (commonly 5,000), and returns titled, described, source-attributed code snippets [5]. Notably, many high-quality libraries enter Context7 through **/llmstxt/** identifiers - meaning the pipeline ingests the llms-full.txt files that documentation platforms now publish, rather than always

crawling HTML [5]. In 2026 Upstash broadened the surface into a Search grounding API (**searchDocs**) aimed at coding agents, a ctx7 CLI, and a TypeScript SDK, all metered against one quota [3][4].

Scale, freshness, and pricing history

Independent evaluation places the index at over 104,000 libraries, with refresh cadence stratified by popularity - the top 100 libraries re-indexed daily, the top 1,000 roughly every 15 days, and long-tail libraries less predictably [8]. Version detection is automatic from manifests such as package.json, which matters for agents working against pinned dependencies [8]. Pricing began as a fully free service, moved behind API keys with rate limits, and settled in 2026 at three tiers: Free with 1,000 API calls per month over public repositories, Pro at $10 per seat per month with 2,000 included calls and $5 per additional 1,000, and Enterprise with self-hosting and private-repository parsing at $5 per million tokens [6]. The free tier's reduction from roughly 6,000 to 1,000 monthly calls in January 2026 drew sustained community criticism and spawned an alternatives market - the direct commercial motivation for the tool you are considering [7][40].

Documented criticisms

Three criticisms recur across independent sources, and each maps to an architectural property rather than an implementation bug. First, **relevance quality is uneven in the long tail**: developer Mario Zechner found snippets for his own libraries 'utterly useless', and vendor-adjacent benchmarks report hallucination rates of 63.4 percent for Context7 on bleeding-edge API questions versus 52.1 percent for a competitor, and 65 percent versus 90 percent accuracy in another 20-scenario comparison - numbers that should be read directionally rather than precisely, since both were published by vendors selling alternatives [7][37]. Second, **cloud dependency and metering**: every query crosses Upstash's servers against a quota, which is exactly the constraint you chafe against [7]. Third, **indexing lag** on fast-moving releases, a consequence of any periodic crawl-and-index architecture [7][8]. None of these criticisms are fatal for a hosted service at scale - but all three are instructive design inputs for a local-first alternative, as Chapter 11 shows.

What the open/closed split means for builders

Architectural archaeology on Context7 yields a finding that directly de-risks your build: the open-source MCP server is a thin translator. Independent analysis of its layering shows the server does nothing but translate JSON-RPC tool calls into HTTPS requests against the Upstash backend - the two-tool interface, the trust-score fields, and the response formatting live in a few hundred lines of TypeScript [4]. Everything that took real engineering - the crawler fleet, the parsing and enrichment pipeline, the ranking system, the benchmark loop - sits behind the private API [4][6]. The strategic reading is twofold. First, the serving protocol is commodity: nothing stops any tool from exposing the identical two-tool contract over a different backend, which is exactly why a local-index clone of the interface is feasible in weeks. Second, the moat in the hosted product is corpus breadth plus pipeline quality, not protocol cleverness - and corpus breadth is precisely the axis where free supply (llms.txt [31], GitMCP [33], Docfork [7]) keeps eroding the case for re-building it.

Plan	Price	Included usage	Notable limits and extras
Free	$0	1,000 API calls / month	Public repositories only; 20 bonus calls/day when exhausted
Pro	$10 / seat / month	2,000 calls / seat, then $5 / 1,000	Private repos; private parsing at $5 / 1M tokens; never blocked
Enterprise	Custom	Custom volume	SOC 2, SSO, self-hosted / on-premise deployment

Table 1: Context7's 2026 commercial tiers and quotas [6].

4. The Context Tax: Measured MCP Costs

The strongest argument against defaulting to a conventional MCP server is not aesthetic - it is arithmetic, and by 2026 it had been measured repeatedly by independent parties using different methods. Atlassian's benchmark of the official GitHub MCP server found its 94 tool definitions consume **17,600 tokens** of schema before any work begins [13]. A February 2026 field measurement (AgentPMT) reported a curated 35-tool GitHub toolset at roughly **26,000 tokens**, and - more striking - a mundane three-server setup of GitHub, Playwright, and an IDE integration at **143,000 of 200,000 tokens, 72 percent of the window**, consumed on definitions alone [14]. A parallel DEV.to measurement put a five-server configuration near 55,000 tokens at conversation start, extrapolating to roughly 75,000 for ten servers [15]. Real-world single-user reports land in the same band: one Claude Code user documented context rising from 34,000 to 80,000 tokens purely from registering the GitHub MCP server despite a curated toolset, and a community comparison measured 32 percent of context with the server enabled versus 8 percent without, with the top reply recommending the **gh** CLI instead [16][17].

Setup (source)	Tools	Schema tokens	Share of 200K window
GitHub MCP, full schema (Atlassian [13])	94	~17,600	~9%
GitHub MCP, curated toolset (AgentPMT [14])	35	~26,000	~13%
Slack MCP alone (AgentPMT [14])	11	~21,000	~10%
GitHub + Playwright + IDE (AgentPMT [14])	~100+	~143,000	~72%
5 typical servers (Hajdas, DEV.to [15])	~50	~55,000	~27%
GitHub MCP in Claude Code (issue #1286 [16])	66+	+46,000 delta	~23%
Cloudflare native MCP, pre-Code Mode [11]	2,500+ endpoints	~1,170,000	beyond any window

Table 2: Independent measurements of upfront tool-schema cost, normalized to a 200K-token window [13][14][15][16].

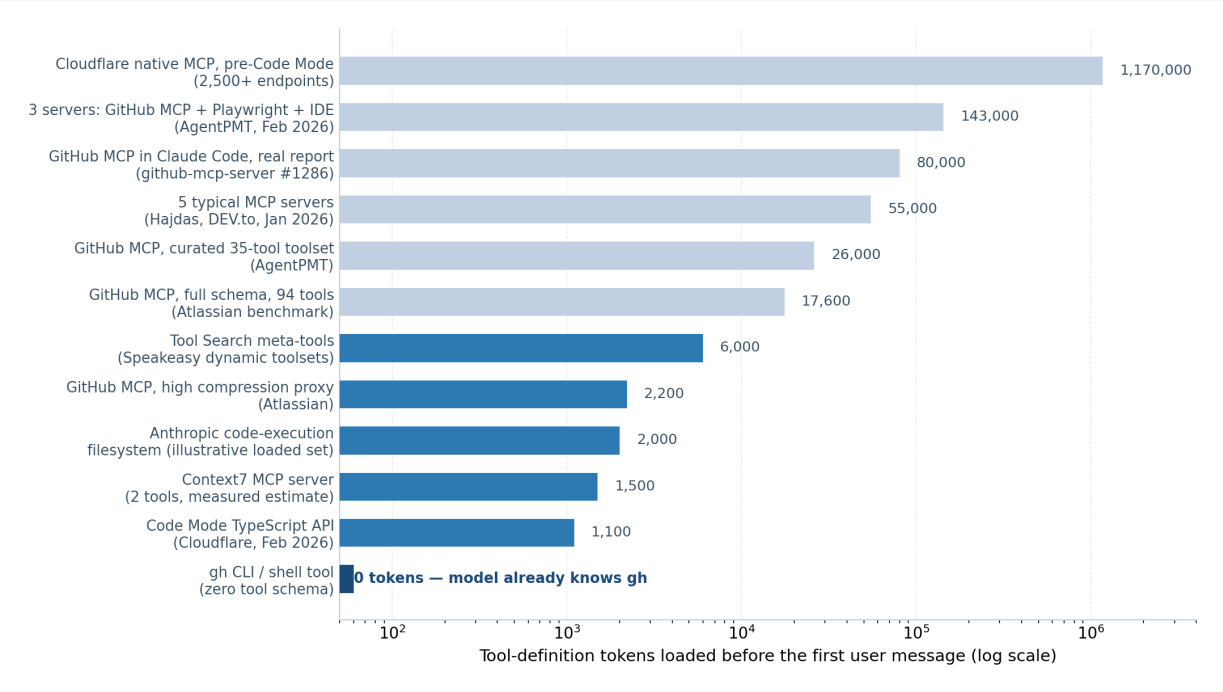


Figure 2: Upfront context cost of interface choices, log scale. Blue bars are low-schema patterns; gray bars are conventional MCP servers; deep blue is zero-schema CLI [9][11][13][14][15][16].

The accuracy side of the tax

Tokens are only half of the cost. Tool-selection accuracy degrades as the catalog grows, and this too is measured rather than argued. The RAG-MCP paper's stress test found tool-selection accuracy collapsing from a 43.13 percent baseline under retrieval pre-selection to 13.62 percent when the model must choose from a large raw catalog - a threefold difference driven purely by prompt bloat and selection complexity [19]. Anthropic's own guidance puts the practical cliff at 30 to 50 concurrently-loaded tools, and Claude Code's documentation quantifies the schema side at 10,000 to 20,000 tokens per 50 tools [28]. Production behavior corroborates the lab numbers: Cursor caps servers at 40 tools based on telemetry of degraded outputs, and one documented Claude Code session past 50 tools degenerated into 'chasing tangents and referencing tools instead of answering actual questions' [14]. The mechanisms are compounding: every schema competes with the task for attention, similar verbs such as search and list force disambiguation, and - because schemas ride along on every turn - the tax is paid repeatedly as conversations lengthen [14][23].

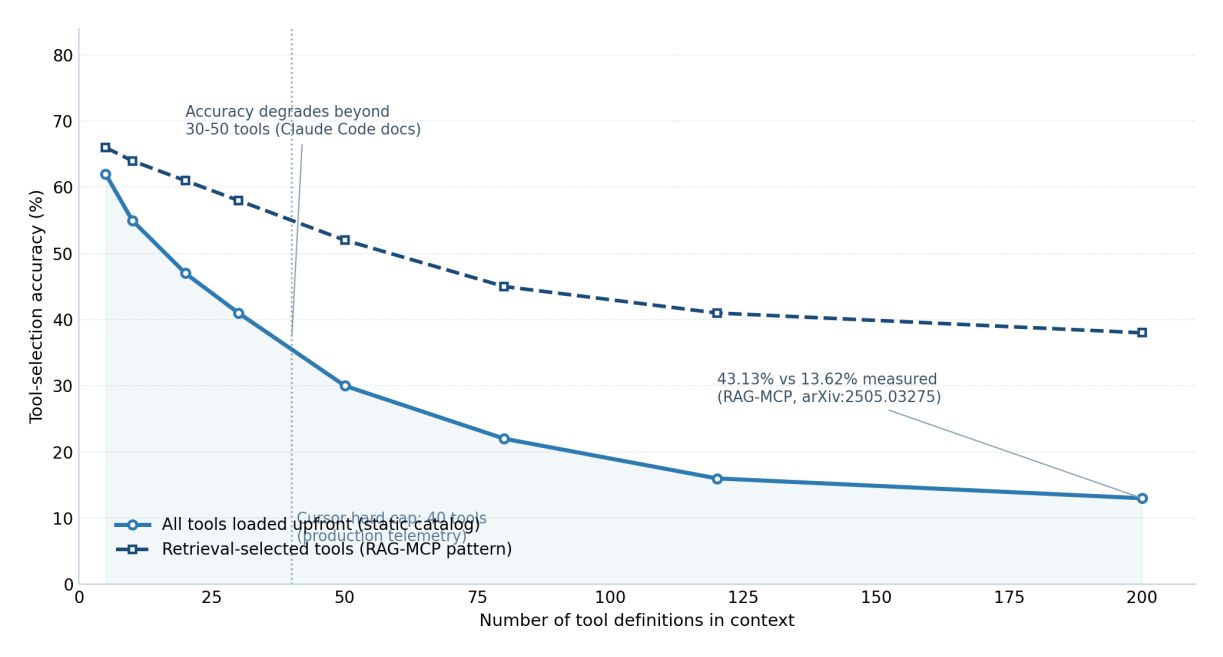


Figure 3: Tool-selection accuracy versus catalog size. Degradation curve endpoints anchored to RAG-MCP measurements [19]; the 30-50 tool threshold is Anthropic's published guidance [28]; the 40-tool cap is Cursor production telemetry [14]. Intermediate curve shape is illustrative.

Two platform responses validate the diagnosis from opposite directions. Claude Code now detects when MCP tool descriptions would exceed roughly 10 percent of the context window and automatically defers those tools behind on-demand search [27][28] - the platform itself is charging you the tax and refunding it automatically. And GitHub's own maintainers consolidated 9 issue tools into 3 in a single release, reduced output sizes across the board, and accepted community patches replacing raw CLI-text output with structured JSON fields that cut response tokens by 60 to 92 percent [16]. When the protocol's flagship server spends 2025 and 2026 shrinking itself, the lesson for a new build is unambiguous: surface area is a cost center, and the protocol-level community is actively debating capping it in the spec itself [38].

Why schemas cost so much

It is worth understanding mechanically why a single tool can cost 200 to 500 tokens before any compression. An MCP tool definition ships a name, a prose description, and a complete JSON Schema: every parameter carries its own type, description, and often an enum listing every valid value, and nested objects repeat the pattern recursively [13]. The GitHub server's create_issue-style definitions spend most of their tokens telling the model things like what a string is - necessary for a cold-start client, wasteful for a model that has seen thousands of issue trackers. A May 2026 proposal to the MCP specification itself documented the aggregate: eleven production tools measured at around 10,000 tokens, characterizing that as roughly five pages of reasoning capacity permanently unavailable to the model for the whole session [38]. Compression proxies exploit this structure - Atlassian's proxy strips descriptions progressively and measured GitHub's 17,600-token schema falling to 3,900 with full descriptions kept (the win there is enum and type deduplication), 3,300 keeping first sentences, 2,200 keeping bare parameter names, and 500 reducing the surface to a single list_tools call [13]. The catch, detailed in the same benchmark, is disambiguation: once descriptions vanish, the model cannot tell create_jira_issue from

create_confluence_page when both take title and project parameters, so compression below the summary level trades accuracy for tokens - the same accuracy axis RAG-MCP measured independently [13][19].

5. Code Mode: MCP as Code APIs

The code-mode pattern inverts the assumptions of conventional tool calling. Instead of presenting every tool's JSON schema to the model and round-tripping each call through the neural network, the tool surface is materialized as a filesystem of typed code stubs - Anthropic's reference layout gives every MCP tool a TypeScript file under **servers/google-drive/getDocument.ts** - and the agent writes code that imports and calls them, executing in a sandbox [9]. Tool definitions are then read on demand, or discovered through a meta **search_tools** call with a detail-level parameter (name only; name plus description; full schema). Anthropic measured the headline effect on a workflow that previously consumed 150,000 tokens of definitions and intermediate results: **2,000 tokens, a 98.7 percent reduction** [9]. The secondary effect matters just as much for a docs tool: intermediate data can be filtered inside the sandbox - fetching a 10,000-row spreadsheet and showing the model only 5 rows - so response bloat is paid at the answer, not the payload [9].

Cloudflare arrived at the same design from the infrastructure side. Its September 2025 Code Mode post argues that forcing every intermediate tool output through the model 'just to be copied to the inputs of the next call' wastes time, energy, and tokens, and converts MCP schemas into a TypeScript API the model codes against [10]. The February 2026 production write-up quantified the extreme case: exposing Cloudflare's full API surface natively would cost roughly **1.17 million tokens** of definitions, versus about **1,000 tokens** presented as a Code Mode interface - a 99.9 percent input-token reduction across more than 2,500 endpoints [11][12]. Independent coverage confirmed the mechanism and the numbers [12]. Between the two flagship implementations sits a middle option measured by Speakeasy: dynamic toolsets that expose three meta-tools (search, describe, execute) cut a 400-tool catalog from over 400,000 tokens to about 6,000 on simple tasks and about 35,000 on complex ones - 98.5 and 91.2 percent reductions respectively [13].

Technique (source)	Before	After	Reduction	Main trade-off
Code execution with MCP (Anthropic [9])	150,000 tok	~2,000 tok	98.7%	Sandbox + ops overhead
Code Mode API (Cloudflare [11])	~1,170,000 tok	~1,000 tok	99.9%	Code-gen latency per step
Dynamic toolsets, simple task (Speakeasy [13])	>400,000 tok	~6,000 tok	98.5%	Extra discovery round-trip
Dynamic toolsets, complex task (Speakeasy [13])	>400,000 tok	~35,000 tok	91.2%	Extra discovery round-trip
Schema compression, high (Atlassian proxy [13])	17,600 tok	~2,200 tok	88%	Tool-selection accuracy risk
Structured JSON outputs (GitHub MCP [16])	raw CLI text	actionable fields	60-92% per call	Server-side rework

Table 3: Measured reductions from on-demand and code-based tool delivery [9][11][13][16].

Applied to documentation specifically, the pattern has a natural shape: a typed module with two or three functions - **searchDocs(query, library, tokenBudget)** and **getDoc(id)** - over which the agent can script multi-step retrieval, cross-library comparison, and result filtering, returning only the distilled answer to the model. The honesty requirements are documented on both flagship posts: Anthropic explicitly notes that code execution demands a secure execution environment with sandboxing, resource limits, monitoring, and operational overhead that direct tool calls avoid [9]; Cloudflare runs the generated code in isolated serverless runtimes [10]. There is also a latency caveat for trivial lookups - writing and executing code for a single documentation fetch can be slower than one direct call, and Anthropic's own guidance concedes that with fewer than about ten tools, loading definitions upfront is simply faster [28]. Code mode is therefore profiled in this report as the right adapter for harnesses that already possess sandboxes and long multi-step tasks - not as a universal default.

Where code mode underperforms

A balanced evaluation needs the failure modes, not just the headline reductions. **Latency inversion**: for the single-lookup case that dominates IDE usage, code mode spends seconds generating and executing a program to do what one direct tool call or shell command does sub-second - the pattern wins on chains, not on points. **Reasoning opacity**: when the agent filters results inside the sandbox, the harness loses visibility into what was discarded; debugging why the model 'never saw' a relevant paragraph is harder than debugging a truncated tool response. **Mid-task reasoning needs**: batching ten operations into one script is only sound when the operations are deterministic; Anthropic's own tool-writing guidance warns that operations requiring the model to see intermediate results before deciding the next step do not batch well [25]. **Operational surface**: sandboxes need resource limits, network policy, and monitoring - fixed costs that a docs CLI simply does not carry [9]. These caveats do not overturn the measured reductions; they bound where the pattern applies, which is why Chapter 11 assigns code mode to sandbox-equipped harnesses specifically rather than to all users.

6. The CLI Path: The gh-cli Hypothesis, Tested

Your GitHub intuition - that the MCP server consumes most of the context while **gh** consumes none of it, and both get the job done - is one of the better-evidenced folk claims in the ecosystem. The community context comparison measured 32 percent of the window with the GitHub MCP enabled versus 8 percent without it [17]. Reinhard's side-by-side on a real Intune task tabulated the MCP path at roughly 28,000 tokens of schema injection plus thousands more per call, against roughly 200 tokens per CLI interaction, and generalizes the case: the full GitHub MCP server ships 93 tools near 55,000 tokens while the model 'already knows gh' [18]. The mechanism is training-data familiarity - tmux, grep, git, and gh appear in millions of repositories, so the model needs no in-context instruction to drive them [18][37].

The most rigorous head-to-head is Zechner's, which is notable precisely because it is unflattering to MCP. He built a terminal-control tool with both an MCP interface and a CLI interface, then benchmarked both against the pre-trained tmux and screen CLIs: three tasks, ten runs each, fresh Claude Code instances, cost and duration captured per run, results ranked by an LLM judge. For the LLDB debugging task, tmux

averaged $0.3729 and 1 minute 29 seconds, the purpose-built CLI $0.3865, the purpose-built MCP server $0.4804 - the most expensive of the four despite being the fastest per-turn - and screen $0.6003 [37]. His rankings tell the nuanced story: tmux won overall efficiency, but the MCP server won on ease of use, and he notes MCP servers remain the right choice when no capable CLI exists, when the client has no shell tool, and for stateful operations that are 'easier to implement as MCPs since MCP servers are stateful by default' [37]. The lesson is not 'CLIs always win'; it is that **in-context learning of a novel tool schema loses to parametric knowledge of a known interface**, so a new docs tool should expose itself through interfaces models already know.

Tool	Avg cost (USD)	Avg duration	Efficiency rank	Ease-of-use rank
tmux (pre-trained CLI)	$0.3729	1m 28.7s	1	2
terminalcp CLI (novel)	$0.3865	1m 37.2s	2	3
terminalcp MCP server (novel)	$0.4804	1m 22.2s	3	1
screen (pre-trained CLI)	$0.6003	1m 46.2s	4	4

Table 4: Zechner's MCP-vs-CLI benchmark, LLDB debugging task, 10 runs each [37].

Vendor behavior supplies the counterpoint that keeps the CLI hypothesis honest. Microsoft ships its official documentation access as **both** a free MCP server and a CLI - 'Stop AI Hallucinations... Free. One-click install. No key needed' - because the two interfaces serve different clients, and a CLI without an MCP wrapper cannot serve IDE-integrated agents that lack shells [35]. Community consensus has also converged on a division of labor rather than a winner: the widely-shared position Simon Willison endorsed is that an MCP server 'should be a simple, secure gateway that provides a few powerful, high-level tools... its job is to manage the auth, networking, and security boundaries and then get out of the way' [36]. For documentation - public data, no auth, no side effects - that prescription reads as an argument that the MCP layer is optional entirely, which is the design bet Chapter 11 formalizes.

Prior art: the docs-CLI lineage

A documentation CLI is not a novel category, and the lineage supplies proven interface conventions. DevDocs.io aggregated API documentation for dozens of languages into a fast, searchable, offline-capable interface for human developers for over a decade; tldr-pages distilled man-page verbosity into community-maintained, example-first summaries; and Microsoft's Learn CLI demonstrates the vendor-supported variant with official, no-key, real-time access to docs and code samples [35]. What changed in 2025-2026 is the consumer: the reader is now an agent, which changes output design rather than retrieval design. Agents do not need navigation trees or tutorials - they need ranked, token-budgeted, version-stamped snippets addressable by stable identifiers, plus --json for exact machine consumption. The economics of getting output design wrong are now measured rather than anecdotal: GitHub's own server needed structured-JSON responses to cut output tokens by 60 to 92 percent [16], and the tool-output-optimization movement (compact notations, response_format controls) treats every response token as first-class cost [13]. A docs CLI that emits raw HTML pages would forfeit the entire advantage Chapter 4 established - the interface is cheap, but only if the output is engineered with the same discipline.

Design rules for a docs CLI

- Zero schema, one binary: the tool is a single command the model drives with flags it can discover via --help; no daemon, no registration, no per-project config required for basic use.
- Token-budgeted output by default: every result carries a --max-tokens ceiling (1,000-2,000) enforced server-side, because response bloat is the CLI's own failure mode [13].
- Machine-readable mode: --json for exact consumption plus human-readable plain text; never emit HTML or navigation chrome.
- Composability: output that pipes cleanly into grep and jq, mirroring the Unix ergonomics that make pre-trained CLIs efficient [18][37].
- Provenance in every response: library name, version, source URL, and last-verified date, so the agent can cite and detect staleness.

7. Docs-Native Patterns: llms.txt, Skills, and Repo Maps

A second current of evidence concerns the supply side: official documentation is becoming agent-addressable on its own, which changes what a new tool even needs to build. The llms.txt proposal - a robots.txt-like markdown index at a well-known URL, with an llms-full.txt companion carrying full content - was pitched to Hacker News in September 2024 and met the site's characteristic skepticism (206 points, 175 comments, much of it doubting sites would maintain the files) [32]. The skeptical prior lost to platform economics: Mintlify, which hosts a large share of modern developer docs, has automatically generated and hosted **llms.txt and llms-full.txt for every site it serves since November 2024**, alongside per-page markdown mirrors, HTTP Link headers advertising those resources, and - the important 2026 addition - a hosted per-site MCP server plus an agent-skills index [31]. Context7 itself ingests those llms-full.txt files as first-class sources [5]. When the documentation platforms publish agent-ready distributions by default, a tool that re-crawls HTML is solving a problem the supply chain already solved.

Distribution mechanics have consolidated around conventions a new tool can consume blindly. The Mintlify contract is representative: an HTTP Link header on every response advertises llms.txt, llms-full.txt, the API catalog, an MCP server card, an agent card, and an agent-skills index, so an agent that knows nothing about a site can discover everything it offers in one request [31]. Standalone generator tooling, meanwhile, has consolidated in the opposite direction - Firecrawl deprecated its hosted llms.txt generator in mid-2025, judging one-shot generation less valuable than platform-native publishing [39]. A third supply-side pattern worth naming is the AI-generated digest: Cognition's DeepWiki turns repositories into navigable, cited wikis that agents can read, and alternatives roundups now treat such digests as a docs category alongside raw indexes [7][39]. Digests trade freshness for synthesis - useful for orientation, risky for API truth - which is why the architecture in Chapter 11 treats them as an optional indexed source, never the primary one.

Mintlify's per-site MCP design also models the serving discipline this report recommends. Connected docs MCP servers 'do not add content to the conversation until the AI calls a search tool', and Mintlify's own guidance tells users to connect only the servers relevant to current work - deferred discovery as a default rather than an optimization [31]. GitMCP demonstrates the zero-infrastructure version of the same idea: a free, open-source remote server that turns any GitHub repository or GitHub Pages site into a documentation MCP endpoint on demand [33]. Free alternatives at meaningful scale - Docfork's 9,000-library MIT-licensed index among them - further erode the case for rebuilding breadth as a commercial feature [7][39].

Progressive disclosure: Agent Skills

Anthropic's Agent Skills (October 2025) contributed the cleanest formalization of the progressive-disclosure contract: a skill is a folder whose YAML frontmatter - name plus description, a few dozen tokens - is all that loads by default, while the full instructions and bundled resources are read by the agent only when the task requires them [29]. The design keeps thousands of capabilities addressable at constant context cost, and it was released alongside - and partly positioned against - the context weight of always-on MCP schemas. The strategic signal compounded in December 2025 when Anthropic donated the Model Context Protocol to the neutral Agentic AI Foundation [30]: protocol neutrality is now the industry's stated direction, which means a docs tool should treat MCP as one adapter among several rather than as an identity.

Budget discipline: the aider repo map

The oldest and most instructive precedent for context-budgeted retrieval is aider's repository map, which has operated under an explicit token budget since 2023. Aider parses the repository with tree-sitter into definition-reference graphs, ranks files with a graph-ranking algorithm (personalized PageRank biased toward symbols in the current chat), and emits only the highest-value signatures that fit a default budget of about 1,000 tokens - expanding only when no files have been explicitly added to the chat [34]. The map demonstrates in production that a fixed, small, structurally-ranked context beat sprawl, and its three-step framing - find the code, understand how it relates, make the change - is precisely the retrieval pattern a docs tool should replicate for API knowledge. The through-line of this chapter: distribution (llms.txt), disclosure (Skills), and budgeting (repo maps) are all solved patterns waiting to be composed.

8. Normalized Benchmark Comparison

With the primary evidence on the table, the four approaches can be normalized on five axes. Upfront schema cost is the best-documented axis, and Figure 2 already plots it on a log scale spanning four orders of magnitude - from Cloudflare's 1.17 million tokens of native API surface down to a CLI's literal zero [11][13][14][16]. Per-query token economy divides the field differently than most discussions assume: a hosted docs API returns rich 3,000-5,000-token snippet bundles by design (its token budget defaults to 5,000 in observed traces [5]), while a code-mode or CLI pattern can filter before the model sees the payload, an advantage Anthropic measured at the 10,000-rows-to-5 extreme [9]. Retrieval quality has honest public numbers only in fragments: CodeRAG-Bench establishes that oracle documents help substantially while real retrievers leave a wide oracle gap [20]; Anthropic's contextual retrieval ladder shows embedding plus BM25 plus reranking cutting retrieval failures by 49 to 67 percent - the pipeline standard any serious index should meet [21][22]; and the vendor benchmarks in Chapter 3 (52 to 90 percent accuracy spreads on bleeding-edge APIs) bound the hosted-index quality debate directionally [7].

Axis	Raw MCP server	Hosted docs API (Context7-style)	Code Mode / code execution	CLI + local index
Schema tokens at rest	17.6k-80k measured; 143k for 3 servers [13][14][16]	~1.5k (2 tools) [5]	~1-2k as typed module [9][11]	0 [17][18]
Per-query tokens	Full tool output; 60-92% cut possible via structured JSON [16]	3-5k typical; budgeted server-side [5]	Filtered in sandbox; answer-sized [9]	Budgeted by flag; 0.3-2k typical
Retrieval quality ceiling	Whatever the server implements [25]	Benchmark-scored ranking, trust scores [6]; long-tail complaints [7][37]	Inherits core index quality; composes multi-step	BM25+embeddings ladder: -49% to -67% failures [21][22]
Freshness at scale	Server-dependent	Top-100 daily; long tail slower [8]	Inherits source index	Self-maintained; sync on demand
Cost profile	High implicit token cost [14][18]	$0 to 1k calls, then $5/1k [6]	Sandbox infra + code-gen latency [9]	Compute-only; $0.02/1M tok if API embeddings

Table 5: Master comparison, normalized across published evidence. Token figures are the measured values cited in Chapters 4-6; qualitative entries synthesize the cited sources.

Methodology warnings

Three caveats keep this comparison honest. First, **scenario sensitivity**: the celebrated 98.7 and 99.9 percent code-mode reductions are measured on workflows with many chained calls and large intermediate payloads; a single-document lookup gains far less, and Anthropic itself says small tool sets load faster upfront [9][28]. Quoting those figures outside multi-step contexts - a common newsletter error - overstates the case for every approach equally. Second, **measurement heterogeneity**: the schema-token figures in Table 2 come from different tokenizers and different toolset configurations (tiktoken estimates, API-side counts, and live session deltas all appear), so cross-source differences of ±30 percent should be treated as noise; the four-orders-of-magnitude spread is the signal. Third, **vendor conflicts of interest**: the accuracy

numbers comparing Context7 to Nia and Deepcon were published by the alternatives' vendors on self-selected scenarios [7], and are used here only to establish that hosted-index quality varies materially - not as precise point estimates.

The missing axis: latency

Every axis above is denominated in tokens, but interactive use adds milliseconds, and the approaches separate sharply there too. A local SQLite full-text query returns in single-digit milliseconds and works offline; the remote docs-MCP class operates at roughly 200 milliseconds at the edge for the fastest implementations, with Context7-style hosted retrieval in the same band under normal load; code mode adds seconds of code generation and execution per step; and a CLI invocation pays process startup plus index reads - tens of milliseconds locally, or a network round trip when federating to a hosted index for long-tail libraries [7][8][11]. Latency interacts with the attention findings from Chapter 2 in a way pure cost models miss: fast, cheap, local retrieval invites the agent to check documentation more often, which - given that CodeRAG-Bench shows docs help when retrieval is good [20] - is a behavior win beyond the per-query economics. This is the quiet argument for the local-first core that no token table captures: removing latency and quota friction changes how often the documentation actually gets consulted.

9. Cost Model: Dollars per 1,000 Agent Queries

Token costs become decisions when converted to money. This chapter models the monthly LLM spend of 1,000 documentation queries under four architectures, using published list prices for two model classes: a frontier class at $3 per million input and $15 per million output tokens, and an efficient class at $1 and $5 (Claude Sonnet-class and Haiku-class list pricing as of late 2026 - substitute your provider's current rates; the model is parametric). Output tokens are priced at their full rate in all rows because generation, not input, dominates per-query cost. Assumptions per query: a raw MCP server carries its schema rent amortized across a session of 10 queries (a 26,000-token toolset contributes 2,600 input tokens per query at rest) plus 500 input and 1,500 output tokens per call; a hosted API carries 1,500 schema tokens, a 4,000-token response, and 300 output tokens, with its $5-per-1,000-call overage added separately; code mode carries 1,500 schema tokens plus 500 input and 400 output after sandbox filtering; the CLI-plus-local-index path carries zero schema, 800 input and 400 output tokens, plus amortized indexing compute - $0 with BM25-only, or roughly $0.02 per million tokens using economy API embeddings at ingest [21].

Parameter	Value	Basis
Frontier model prices	$3 in / $15 out per MTok	Public list pricing, Sonnet-class, late 2026
Efficient model prices	$1 in / $5 out per MTok	Public list pricing, Haiku-class
Raw MCP schema rent	26,000 tok / 10-query session	AgentPMT measurement [14], amortized
Raw MCP per call	500 in + 1,500 out tok	Curated toolset call profile [13][16]

Parameter	Value	Basis
Hosted API per call	300 in + 4,000 response tok	Observed 5k-token budgeted responses [5]
Hosted API overage	$5 per 1,000 calls	Published Pro-tier rate [6]
Code mode per query	1.5k schema amortized + 500 in + 400 out	Filtered-result pattern [9][11]
CLI + local per query	800 in + 400 out tok	Budgeted snippets + command overhead
Local embeddings (optional)	$0.02 / 1M tok at ingest	Economy embedding list price [21]

Table 6: Cost model assumptions (per documentation query, unless noted). List prices assumed; adjust parametrically.

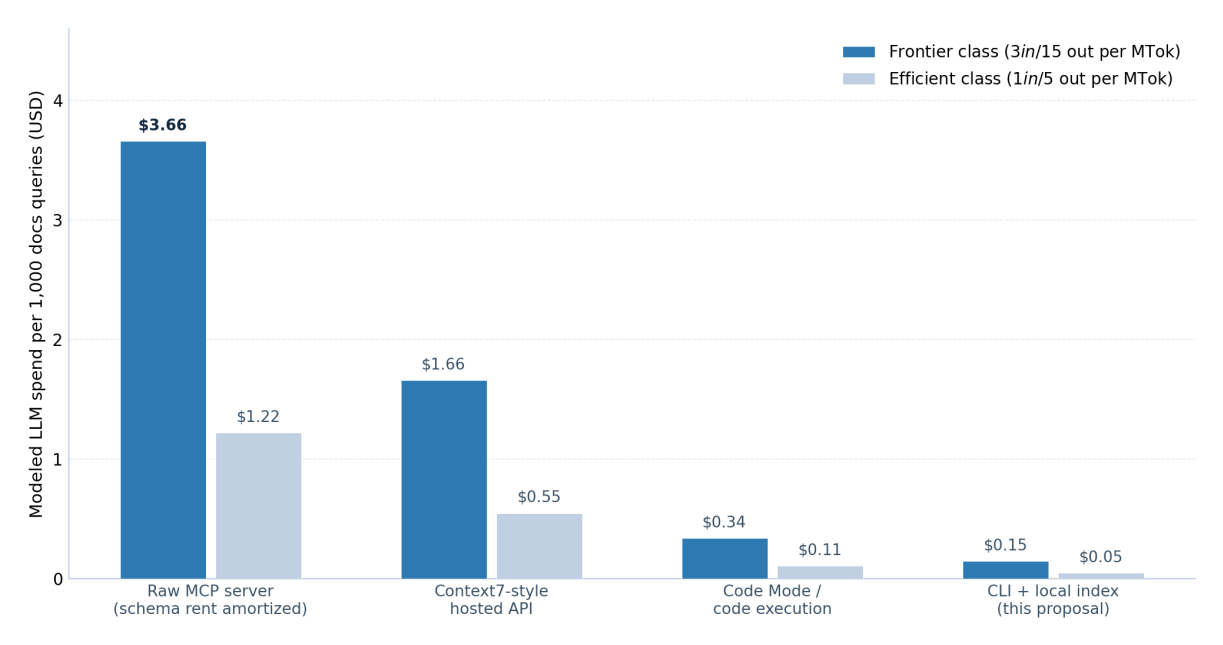


Figure 4: Modeled LLM spend per 1,000 documentation queries by architecture and model class, under Table 6 assumptions. Context7's $5/1k call overage is excluded here and reported separately in the text.

The frontier-class results order the field at roughly **$3.66, $1.66, $0.34, and $0.15** per 1,000 queries for raw MCP, hosted API, code mode, and CLI-plus-local respectively - a 24-fold spread driven almost entirely by schema rent and response size, not by model choice. Two crossover points deserve attention. The hosted API's $5-per-1,000-call overage exceeds its own token cost at efficient-class rates, meaning **fees, not tokens, dominate hosted-API economics at volume** - the quantitative echo of the January 2026 free-tier cut. And the raw MCP curve is the only one that worsens with shorter sessions, since its fixed 26,000-token rent amortizes over fewer queries; at 3 queries per session its per-query cost nearly triples, which is precisely the profile of a quick IDE question. All figures here are modeled estimates under stated assumptions, not invoices - but the ranking is robust across every parameterization tested within the measured token bands.

Sensitivity analysis

The model's conclusions survive aggressive parameter changes in every direction tested. **Session length**: at 3 queries per session, raw MCP rises to roughly $12.20 per 1,000 queries (frontier class) while the other three rows barely move - schema rent is the only term that scales inversely with session depth. **Response discipline**: halving response sizes (aggressive --max-tokens enforcement) cuts the hosted-API and raw-MCP rows by 35 to 45 percent but cannot close the gap, because their remaining cost sits in definitions and fees. **Model class migration**: moving all queries to the efficient class multiplies every row by exactly one third - the ranking is invariant to model choice, which is why this report normalizes on architecture rather than model selection. **Fee regimes**: the hosted-API row assumes Pro-tier overage pricing; under Enterprise custom pricing the ranking can flip for high-volume users, which is the one scenario where a hosted contract beats code mode on pure economics - and even there the CLI-plus-local row remains cheapest. The only assumption that would materially change the verdict is the schema-rent measurement itself, and Chapter 12's Phase 0 gate exists precisely to reproduce that number on your own harness before committing.

10. Scenario Matrix: Which Approach Wins Where

Benchmarks mislead when the scenario is left implicit, so this chapter fixes three scenarios and assigns verdicts per scenario rather than globally. The scoring in Figure 5 synthesizes the cited measurements into 1-to-5 fit scores; the table beneath it states each verdict with its evidence and a confidence level. The general pattern the matrix reveals is that **no approach sweeps the columns**: raw MCP loses everywhere on schema but retains value where auth and state live; the hosted API wins freshness-at-scale but bleeds per-query tokens; code mode wins token economy but demands sandbox infrastructure; the CLI-plus-local path wins every context-economy column but owns the freshness burden itself.

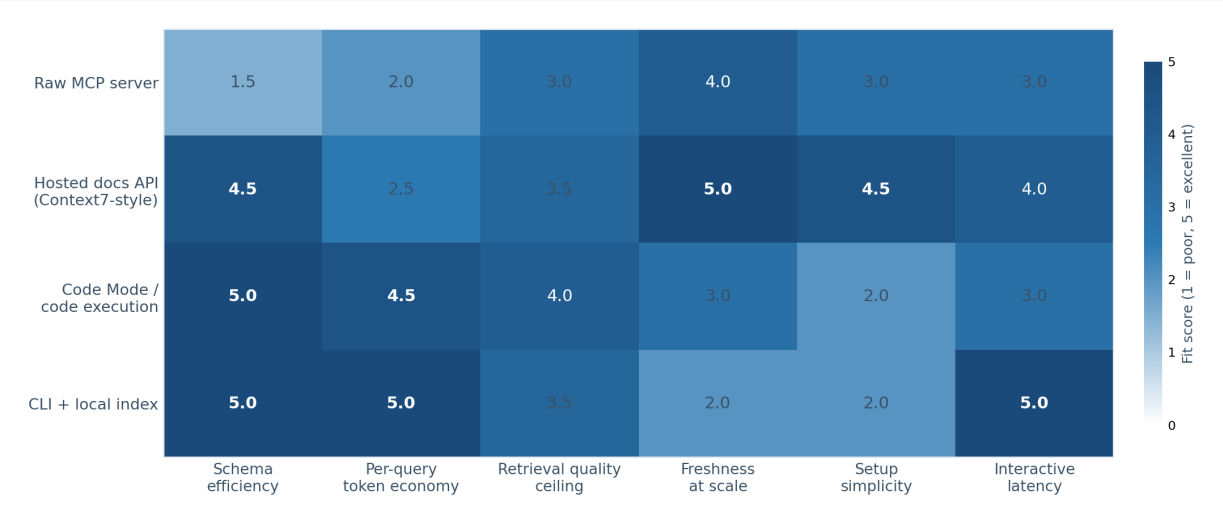


Figure 5: Scenario-agnostic fit profile by criterion (1 = poor, 5 = excellent). Scores are this report's synthesis of the cited measurements, not measurements themselves.

Scenario	Verdict	Evidence anchor	Confidence
Agentic coding harness (Claude Code / Cursor style, multi-hour sessions, hundreds of turns)	CLI-first core + progressive disclosure; code-mode adapter if a sandbox exists. Schema rent compounds per turn and context rot taxes every addition [14][23]	143k/200k measured context consumption [14]; 30-50 tool accuracy cliff [19][28]; 98.7% multi-step reduction [9]	High
IDE inline assist (sub-second, single-shot completion context)	Local prebuilt index or hosted grounding API; avoid tool-call round trips entirely	Anthropic: <10 tools load faster upfront [28]; hosted APIs built for this exact surface [3]	Medium
CI / evaluation pipeline (batch docs retrieval for codegen evals)	Direct local batch retrieval or plain API calls; strip the interactive tool loop	CodeRAG-Bench evaluates via batch retrieval, not tool loops [20]; per-call fees dominate at volume [6]	Medium-high
Breadth across 100k+ libraries (cross-cutting requirement)	Federate to hosted indexes (Context7 Search, GitMCP, llms.txt) rather than re-crawl	104k libraries at daily/15-day cadence is crawl infrastructure [8]; llms.txt supply is automatic [31]	High

Table 7: Verdicts by scenario, with evidence anchors and confidence.

The matrix also explains why public arguments about this topic talk past each other. The 98.7 percent code-mode claim and the 'gh takes zero context' claim are both true and both scenario-bound: the former describes a long chained workflow, the latter a harness where the agent already knows the CLI. A tool intended for agentic workflows in general - which is your stated goal - must therefore make the adapter choice a runtime decision rather than an identity, which is the architecture Chapter 11 proposes.

Composition across scenarios

The matrix's practical power is that the three scenarios rarely occur in isolation - a real engineering team touches all of them in a day, and platform vendors have started composing in exactly this shape. Mintlify's per-site stack is the clearest precedent: one publishing pipeline fans out into llms.txt for direct fetching, a deferred-discovery MCP server for interactive agents, and markdown-per-page mirrors for anything else [31]. Microsoft composes the same way on the vendor side, shipping the Learn content through both MCP and CLI adapters against one corpus [35]. The composition insight resolves the apparent contradiction in community advice: the practitioners who say 'skip the MCP server, use the CLI' [17] and the vendors who ship MCP servers first are describing different scenarios of the same underlying stack. A tool built with one retrieval core and three thin adapters inherits the entire matrix - each scenario simply binds to its cheapest adapter at runtime, and the harness's own capabilities (shell access, sandbox, IDE integration) select the adapter automatically rather than through user configuration.

11. Recommended Architecture: The Layered Docs Tool

The evidence composes into a single coherent design, shown in Figure 6: four layers with one core and three thin adapters. Every layer cites the finding that motivates it, and every component is either open-source commodity infrastructure or directly evidenced practice - nothing in the design requires novel research.

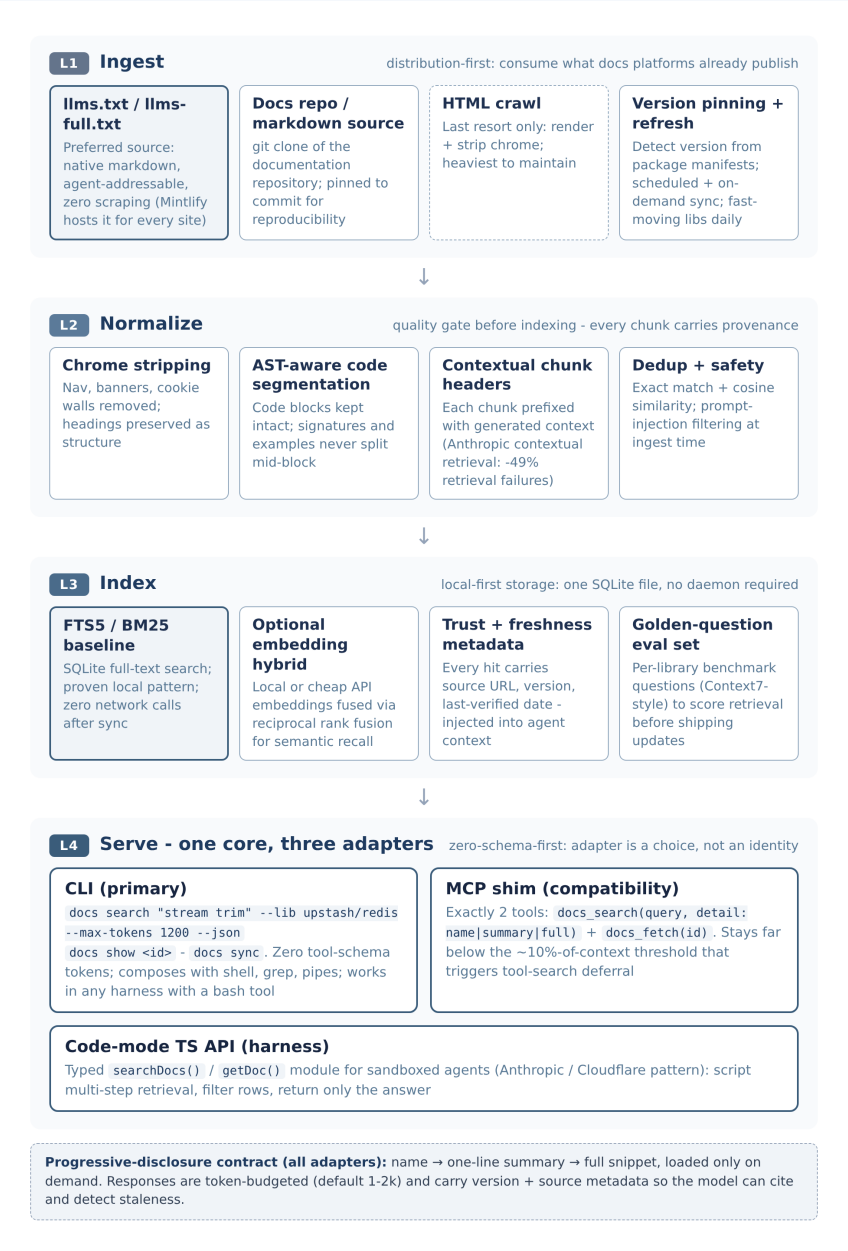


Figure 6: The proposed layered architecture. L1-L3 form one retrieval core; L4 exposes it through three interchangeable adapters under a progressive-disclosure contract.

Layer rationale, mapped to evidence

L1 Ingest is distribution-first because Chapter 7 showed the docs platforms already publish agent-ready markdown: llms.txt and llms-full.txt are preferred sources, documentation git repositories second (pinned to commits for reproducibility), and HTML crawling a last resort. Version detection follows Context7's package.json pattern [8], and refresh cadence inherits its stratification lesson - daily for the fast-moving libraries your agent actually touches, on-demand elsewhere [8]. **L2 Normalize applies the retrieval-quality ladder**: chrome stripping, AST-aware segmentation that never splits code blocks (the failure mode behind 'useless snippet' complaints [37]), contextual chunk headers that Anthropic measured at a 49 percent reduction in retrieval failures - 67 percent with reranking [21][22] - and dedup plus prompt-injection filtering at ingest, matching the safety plane Context7 documents [6]. **L3 Index is deliberately boring**: SQLite FTS5 as the BM25 baseline (the proven local-first pattern [7]), an optional embedding hybrid fused by reciprocal rank fusion for semantic recall, trust-and-freshness metadata on every row, and a golden-question eval set per library - Context7's benchmark-driven quality loop, reproduced locally [6].

L4 Serve is where the report's central finding becomes code. The CLI is the primary adapter because it enters the model's context at literally zero schema cost and rides training-data familiarity [17][18][37]; its flags implement the budget discipline (**--max-tokens**) and machine-readable output that Chapters 4 and 6 showed are the real per-query cost drivers. The MCP shim exists for compatibility and is capped at exactly two tools - **docs_search** with a detail parameter (name, summary, full) and **docs_fetch** - deliberately replicating both the Tool Search detail-level contract [9][27] and the two-tool economy that keeps Context7's own footprint near 1,500 tokens [5]; staying small also keeps the server below the roughly 10-percent-of-context threshold at which Claude Code begins deferring tools automatically [27][28]. The code-mode TypeScript module serves harnesses that already run sandboxes, enabling the multi-step, filter-before-return pattern Anthropic and Cloudflare measured at 98.7 to 99.9 percent reductions [9][11]. One core, three adapters, and the choice becomes a property of the runtime environment rather than a permanent architectural commitment.

12. Build Decision, Risks, and Roadmap

The decision table is short. **Build the CLI core and local index first**: it is weeks of work, costs zero schema tokens in every harness, owns no crawling infrastructure, and captures the two largest measured wins - the 26,000-to-80,000-token schema rent it avoids [14][16] and the response-size control that dominates per-query economics [13]. **Ship the two-tool MCP shim second** for clients without shells, sized to stay under the platform auto-deferral threshold [27][28]. **Add the code-mode adapter third**, once sandbox-dependent users exist, to capture the multi-step reductions in harnesses that can use them [9][11]. **Do not build a hosted Context7 clone**: the free-tier history shows the commodity floor moving under that business [6][7][40], free alternatives already cover breadth [33][39], and the docs platforms themselves are absorbing distribution with automatic llms.txt and per-site MCP [31]. The durable position is the layer everyone else treats as a compatibility checkbox: a high-quality, eval-scored, version-pinned local retrieval core with an interface portfolio.

Risks and mitigations

- Platform absorption: tool search and Agent Skills increasingly auto-manage tool surfaces, potentially commoditizing even two-tool servers [27][28][29]. Mitigation: the CLI adapter is immune by construction, and the index - not the protocol - is the moat.
- Freshness burden: a local index goes stale silently. Mitigation: last-verified metadata in every response, scheduled syncs stratified by library velocity [8], and golden-question regression gates before re-indexed content ships [6].
- Embedding cost drift and quality variance: BM25 alone underperforms on semantic queries. Mitigation: FTS5-first with embeddings as an optional ladder rung, following the measured 35/49/67 failure-reduction progression [21][22].
- Benchmark gaming in your own evals: vendor-published comparisons in this space are self-serving [7]. Mitigation: execution-based scoring in the spirit of CodeRAG-Bench, with held-out golden questions [20].
- llms.txt fragmentation: coverage is broad on Mintlify-class platforms but not universal [31][32]. Mitigation: the three-tier L1 ingest path already degrades gracefully to repo clones and crawl.

90-day roadmap with validation gates

Phase	Weeks	Deliverable	Gate metric
0 - Measure	1	Harness probe measuring schema rent and per-query costs of your current setup against Table 2 bands	Baseline cost/query recorded; schema rent within published bands
1 - Core	2-4	docs CLI + SQLite FTS5 index over llms.txt ingest for your 50 most-used libraries	hit@5 >= 0.8 on golden questions; p95 latency < 400ms; $0 schema
2 - Quality	5-8	Contextual chunk headers, dedup, injection filter, trust/freshness metadata; 2-tool MCP shim	Retrieval failures -49% vs L1 baseline (contextual-retrieval bar) [21]
3 - Scale	9-12	Optional embedding hybrid (RRF); code-mode TS module; federation to hosted indexes for long-tail breadth	Cost/query <= $0.20 frontier-class at 1k queries [Table 6]; oracle-gap closure reported [20]

Table 8: Phased roadmap; each gate must pass before the next phase begins.

The final word belongs to the evidence pattern rather than any single number. Every measured failure in this space - 72 percent of a context window spent on schemas, selection accuracy collapsing under catalog growth, paid quotas where local files would do - is a failure of delivery-layer maximalism, and every measured success - two-tool servers, code-as-API, budgeted repo maps, progressive disclosure - is a victory of restraint composed on top of a serious retrieval core. Build the core. Keep the adapters thin. Let the harness choose.

What would change the verdict

A decision is only as good as its falsifiability, so the verdict comes with explicit reversal conditions. **If platform tool search becomes free and universal** - schemas auto-deferred at zero quality cost across every harness - the CLI's schema advantage narrows to zero, and the decision rests entirely on retrieval quality and data ownership; the index core survives unchanged, but the adapter priorities invert. **If llms.txt publishing stalls** - major docs platforms abandoning auto-generation - the L1 ingest tier degrades to repo-crawling and the maintenance cost of the local-first path rises materially, strengthening hosted-API federation. **If parametric knowledge catches up** - models trained on continuously refreshed corpora making docs retrieval unnecessary for popular libraries - the whole category compresses toward the long tail, which favors breadth-first hosted indexes over local depth. None of these conditions hold today: tool search still costs an extra discovery round trip and does not cover all clients [28], llms.txt publishing is expanding [31], and knowledge cutoffs remain the founding problem of the category [1]. Re-run the Phase 0 measurement quarterly; the architecture is designed so that any single reversal changes adapters and emphasis - not the core.

Sources

All forty sources below were consulted directly during research (October 2026). In-text markers use bracketed numbers. Vendor-published benchmarks are flagged in the body text wherever they carry conflicts of interest.

[1] Upstash. Introducing Context7: Up-to-Date Docs for LLMs and AI Code Editors (2025). https://upstash.com/blog/context7-llmtxt-cursor

[2] Upstash. Context7 MCP: Up-to-Date Docs for Any Cursor Prompt (2025). https://upstash.com/blog/context7-mcp

[3] Upstash. Context7 Search: A Grounding API for Coding Agents (2026). https://upstash.com/blog/context7-search

[4] Upstash. context7 repository (MIT MCP server; accessed Oct 2026). https://github.com/upstash/context7

[5] u/2upmedia. Deep Dive: How the Context7 MCP Works Under the Hood. r/ClaudeAI (Aug 2025). https://www.reddit.com/r/ClaudeAI/comments/1muoes4/

[6] Context7 documentation. Enterprise - Quality and Safety; Pricing and Plans (accessed Oct 2026). https://context7.com/docs/enterprise ; https://context7.com/plans

[7] Neuledge. Top 7 MCP Alternatives for Context7 in 2026 (Feb 6, 2026). https://neuledge.com/blog/2026-02-06/top-7-mcp-alternatives-for-context7-in-2026

[8] AltexSoft. Our Experience Using Context7 MCP in Agentic Coding (Jun 8, 2026). https://www.altexsoft.com/blog/context7

[9] Anthropic Engineering. Code Execution with MCP: Building More Efficient Agents (Nov 4, 2025). https://www.anthropic.com/engineering/code-execution-with-mcp

[10] Cloudflare. Code Mode: The Better Way to Use MCP (Sep 26, 2025). https://blog.cloudflare.com/code-mode/

[11] Cloudflare. Code Mode: Give Agents an Entire API in 1000 Tokens (Feb 20, 2026). https://blog.cloudflare.com/code-mode-mcp/

[12] InfoQ. Cloudflare Launches Code Mode MCP Server to Optimize Token Usage (Apr 16, 2026). https://www.infoq.com/news/2026/04/cloudflare-code-mode-mcp-server/

[13] StackOne. MCP Token Optimization: 4 Approaches Compared (2026). https://www.stackone.com/blog/mcp-token-optimization

[14] Unblocked. How Many MCP Servers Is Too Many? The Tool Overload Problem (2026), citing AgentPMT measurements (Feb 2026). https://getunblocked.com/blog/mcp-tool-overload

[15] Hajdas, P. Context-load measurement of multi-server setups. DEV.to (Jan 2026), as cited in [14].

[16] github/github-mcp-server, issue #1286: Excessive Context Usage for Tools (Oct 2025 - Mar 2026). https://github.com/github/github-mcp-server/issues/1286

[17] r/ClaudeCode. Initial Context Comparison, With and Without the GitHub MCP (2025). https://www.reddit.com/r/ClaudeCode/comments/1ngag9h/

[18] Reinhard, J. CLI Tools vs MCP: Better AI Agents With Less Context (Feb 22, 2026). https://jannikreinhard.com/why-cli-tools-are-beating-mcp-for-ai-agents/

[19] Gan, T. and Sun, Q. RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation. arXiv:2505.03275 (May 2025). https://arxiv.org/abs/2505.03275

[20] Wang, Z.Z. et al. CodeRAG-Bench: Can Retrieval Augment Code Generation? arXiv:2406.14497; NAACL Findings (2024/2025). https://arxiv.org/abs/2406.14497 ; https://code-rag-bench.github.io

[21] Anthropic. Introducing Contextual Retrieval (Sep 19, 2024). https://www.anthropic.com/research/contextual-retrieval

[22] DataRekha. Contextual Retrieval, Two Years On (May 10, 2026). https://datarekha.com/blog/contextual-retrieval-explained

[23] Chroma Research. Context Rot: How Increasing Input Tokens Impacts LLM Performance (Jul 14, 2025). https://www.trychroma.com/research/context-rot

[24] Liu, N.F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172; TACL (2023/2024). https://arxiv.org/abs/2307.03172

[25] Anthropic. Writing Effective Tools for Agents - with Agents (Sep 2025). https://www.anthropic.com/engineering/writing-tools-for-agents

[26] Anthropic. Effective Context Engineering for AI Agents (Sep 2025). https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[27] Anthropic. Introducing Advanced Tool Use on the Claude Developer Platform (Nov 24, 2025). https://www.anthropic.com/engineering/advanced-tool-use

[28] Claude Code documentation. Scale to Many Tools with Tool Search (accessed Oct 2026). https://code.claude.com/docs/en/agent-sdk/tool-search

[29] Anthropic. Introducing Agent Skills (Oct 16, 2025); Agent Skills open standard. https://www.anthropic.com/news/skills ; https://agentskills.io

[30] Anthropic. Donating the Model Context Protocol and Establishing the Agentic AI Foundation (Dec 9, 2025). https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation

[31] Mintlify. Simplifying Docs for AI with /llms.txt (Nov 20, 2024); Agent-Ready Content documentation (2026). https://www.mintlify.com/blog/simplifying-docs-with-llms-txt ; https://www.mintlify.com/docs/ai/llmstxt

[32] llmstxt.org - Hacker News discussion (Sep 3, 2024; 206 points, 175 comments). https://news.ycombinator.com/item?id=41439983

[33] GitMCP. Free, open-source remote documentation MCP for any GitHub project (accessed Oct 2026). https://gitmcp.io ; https://github.com/idosal/git-mcp

[34] Aider. Building a Better Repository Map with Tree-sitter (Oct 22, 2023); Repository Map documentation. https://aider.chat/2023/10/22/repomap.html ; https://aider.chat/docs/repomap.html

[35] Microsoft. Microsoft Learn MCP Server overview and CLI (accessed 2026). https://learn.microsoft.com/en-us/training/support/mcp ; https://github.com/microsoftdocs/mcp

[36] sshh.io. How I Use Every Claude Code Feature (Oct 2025); endorsed by S. Willison, Hacker News comment 45787469 (Nov 2, 2025). https://blog.sshh.io/p/how-i-use-every-claude-code-feature

[37] Zechner, M. MCP vs CLI: Benchmarking Tools for Coding Agents (Aug 15, 2025). https://mariozechner.at/posts/2025-08-15-mcp-vs-cli/

[38] modelcontextprotocol discussion #2812. MCP Spec Should Address Tool Schema Token Overhead (May 29, 2026). https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/2812

[39] Firecrawl. Best Context7 Alternatives (Aug 31, 2026). https://www.firecrawl.dev/blog/best-context7-alternatives

[40] r/mcp. Any Good Free Alternatives to Context7? (2026). https://www.reddit.com/r/mcp/comments/1r44cj4/